Pull Requests & Merging in GitHub

Data Science Bootcamp

NASHVILLE SOFTWARE SCHOOL

Objectives

1. Be able to create a Pull Request when your code is ready for review
2. Be able to respond to a Pull Request from a teammate
3. Be able to merge your branch back to main then delete your branch
4. Be able to complete pull the updated code from the main branch to your local computer

Branches in GitHub Refresher

- Branches are used when multiple people are working from the same repository
- This allows each person to push their code while they're working on a task without messing with the code on the main branch
- This is very important when working on complex projects where each person is focused on their own tasks within the larger project
- To reduce confusion, create the branch for the task you're working on **BEFORE** starting that work
- We will now shift into the process for reviewing completed code, merging it into the main branch, and pulling those updates to the main branch to each person's computer

A Few Important Notes on Naming

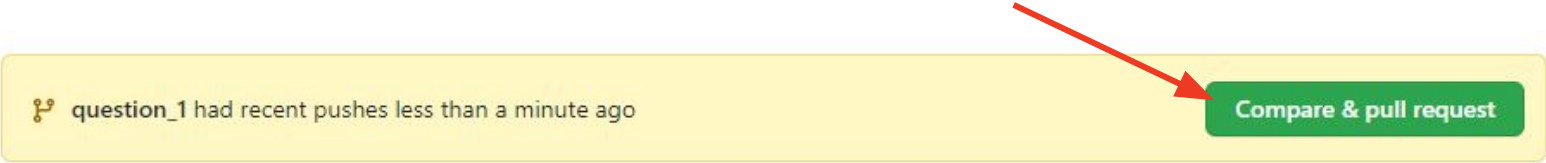
- To keep things organized and avoid duplication, your team needs to agree on conventions for naming branches and files
- There may be instances when everyone on the team wants to attempt the same question, so the naming convention should allow for this:
  - The branch names could be **question_1_initials**
  - The script/file names could be q1_initials
- Setting this at the beginning of the project before anyone starts working will save a ton of time and confusion as the project moves forward
- Duplication of file or branch names can create merge conflicts, and those are a headache to untangle in GitHub
- Consistent naming conventions also make for a much cleaner final repository

GitHub Pull Requests

- Before beginning work, create a branch for your issue/task and push to that branch regularly as you make progress
- When you have completed the code for an issue/task in GitHub, you will move that issue to the In Review column on the Project's Kanban board
- After you have pushed your final code to the branch for that issue, you will then create a Pull Request
- Pull Requests serve a few purposes:
  - Give teammates an easy way to review code, make comments, and ultimately approve work
  - Once reviews are completed and the code is approved, you can then easily merge your code with the main branch and delete the branch associated with the completed issue
  - If you link your Pull Request with the relevant issue on the Project's Kanban board, it will automatically move the issue into the Done column

Creating a Pull Request

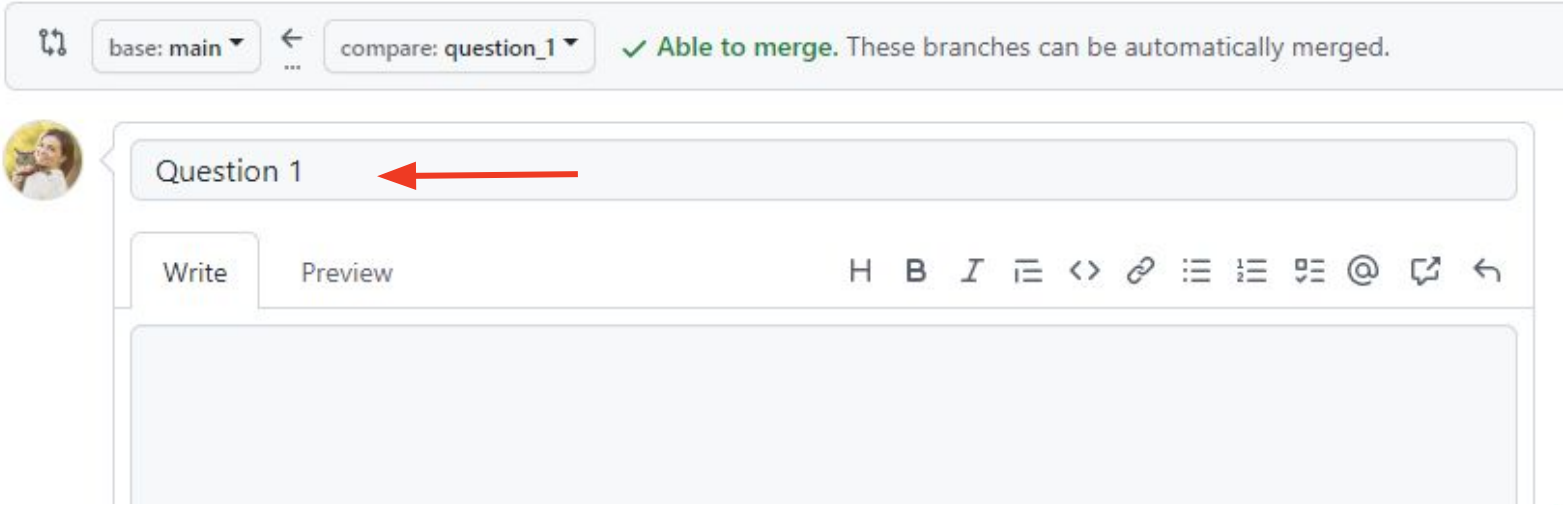
Once you've pushed your final code to your branch, navigate to your team's repository on GitHub, and you should see something like this:



Click on "Compare & pull request" to create a Pull Request for your code.

Creating a Pull Request

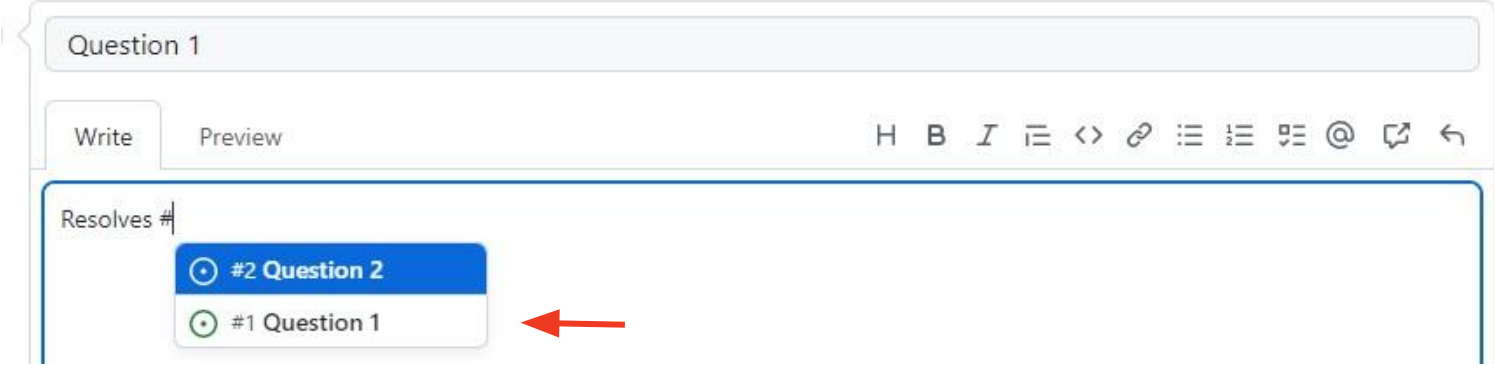
The screen should look like this:



Sometimes the commit message you entered when you pushed will go in the title field, but you can edit it if you wish

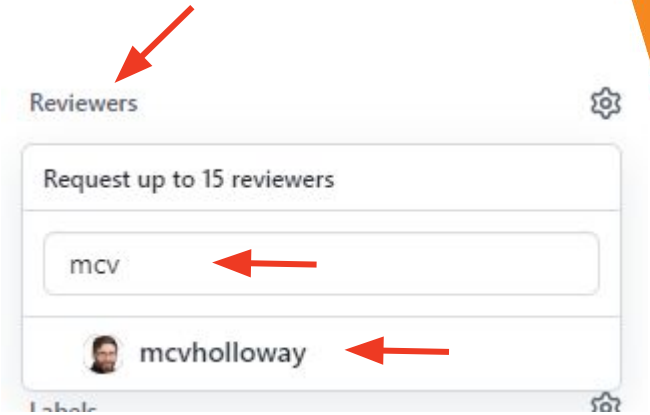
Creating a Pull Request

In the comments field, if you type the hashtag, you will see a popup to connect the Pull Request with the relevant issue from the Kanban board. Select the appropriate issue then add any additional notes you would like to include.

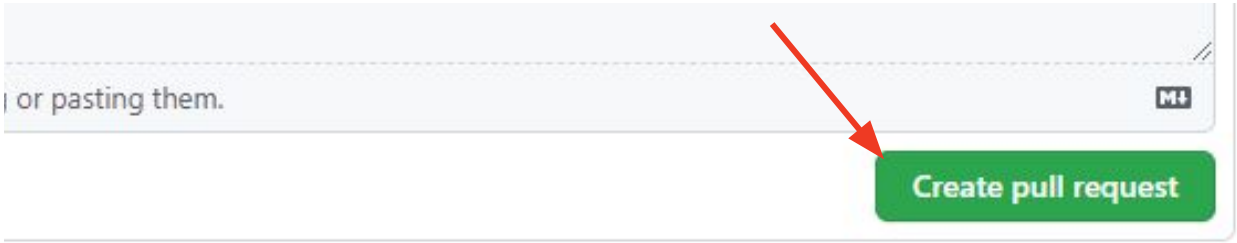


Create a Pull Request

Next, assign your teammates as Reviewers along the right side of the page. If you start typing their username, they'll come up and you can click on them

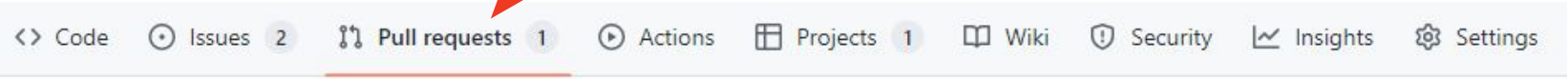


Finally, click "Create pull request"

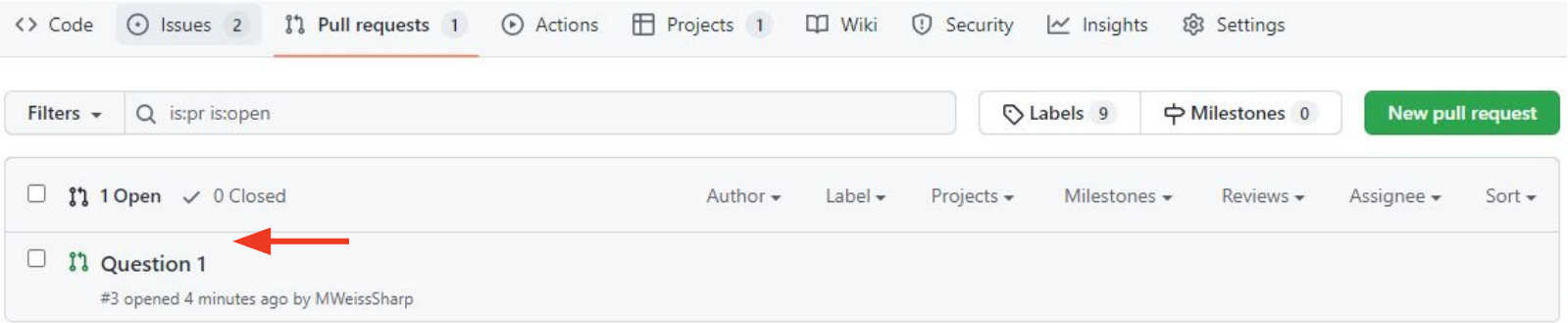


Review a Pull Request

To see all available pull requests, navigate to the project's repository, then click on the "Pull requests" tab



Then click on the pull request you want to review

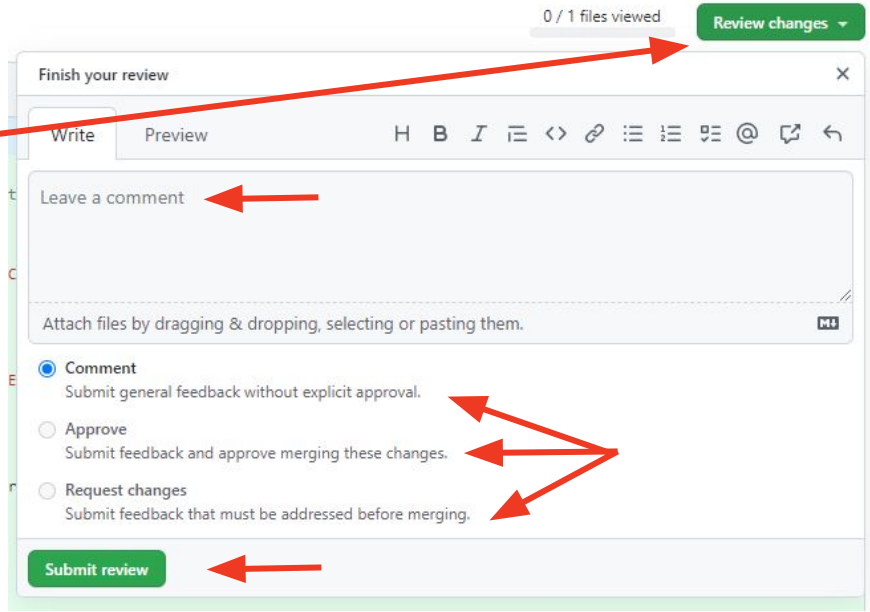


NASHVILLE SOFTWARE SCHOOL

Review a Pull Request

Within the pull request, click on the “Files changed” tab

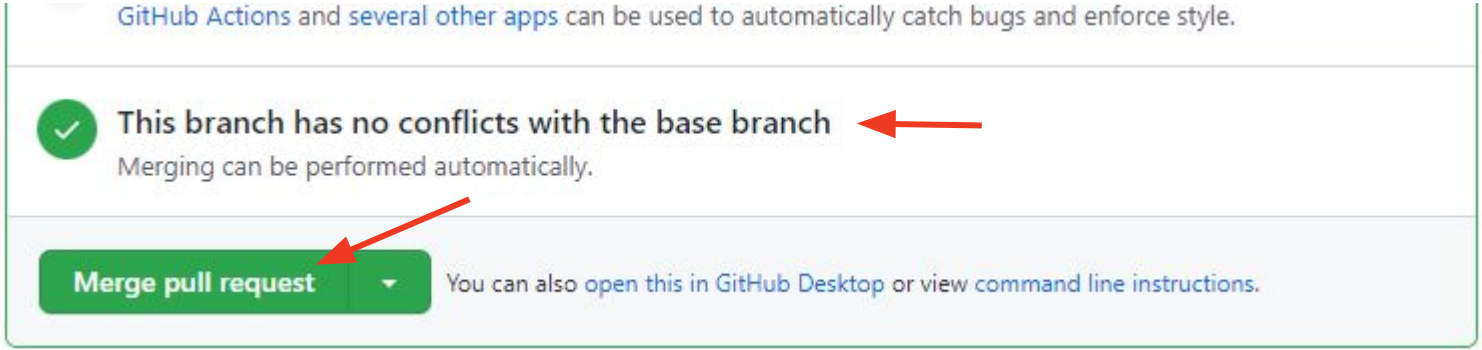
Once you have reviewed the code, click on “Review changes” so that you can add any comments, select the appropriate status (Comment, Approve, or Request changes), the click “Submit review”



Merge a Pull Request

Once all of your teammates have reviewed your code, you have completed any changes/updates, then received approval from all members, you will be ready to merge your code back into the main branch.

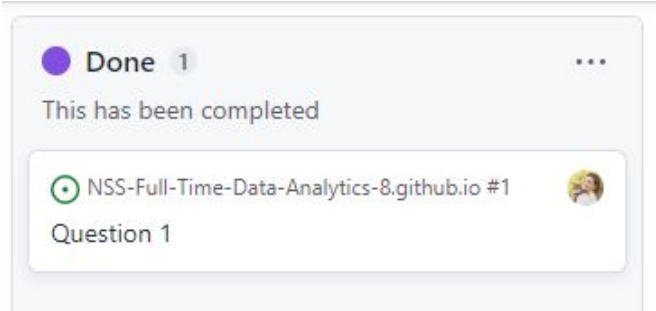
During this class, each person will merge their own code once all approvals are complete. To do this, click into the pull request, scroll down the page a bit, confirm that you see the message that there are no conflicts with the base branch, then click "Merge pull request"



Merge a Pull Request

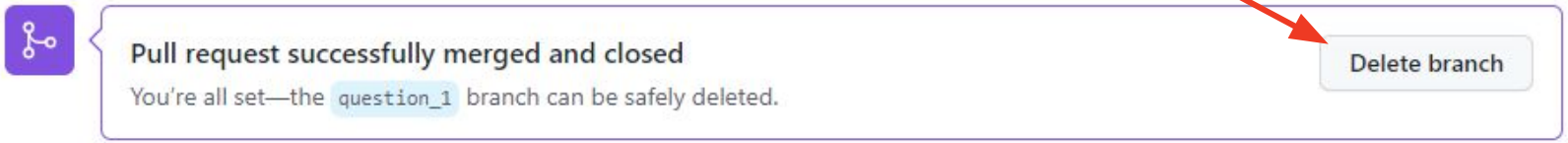
A confirmation will pop up. Look it over to make sure this is the correct code that you want to merge to main, then click “Confirm merge”

Confirm the associated issue has moved to the Done column or move it to that column if it hasn’t



Delete the Branch

Once you have merged your branch back to main, you should see the message below come up. The final step of the merging process is to delete that branch, so go ahead and click “Delete branch”



Pull from the Updated Main Branch

To avoid potential merge conflicts, once anyone on the team merges a branch back into main, no one else should create a new branch until AFTER they have pulled that updated code from the main branch.

To do this, you will need to switch to the main branch, and execute the pull command. These are the commands you would enter:

1. `git checkout main`
2. `git branch` (just to confirm you have switched to the main branch)
3. `git pull origin main`

From here, you can select your next issue/task from the Kanban board, create a new branch, and get back to coding!

Team Project Workflow

1. Clone down the repo
2. One person creates the Kanban board and creates the initial issues/tasks
3. Agree on naming conventions for branches and files to avoid name duplication
4. Once each person is assigned their first issue/task, they each create a branch with a relevant name **BEFORE starting any work** and push to their own branch as they make progress
5. When code is complete for an issue/task and the final code is pushed to its branch, the team member creates a pull request, assigns other team members to review
6. After other members review **and approve**, the team member merges and deletes their branch, confirms the issue/task is in the Done column of the Kanban board
7. When a new branch needs to be created to start work on the next issue, if a merge has occurred, first switch to the main branch and pull the updated code
8. Create the new branch **before starting work related to the next issue**, and the process loops back to step 4